# DSSP PA 02 Report

20261099 양예진

# I. 배경 지식 정리

## 1. Windowing

LTI 시스템의 출력을 구할 때, 시간 도메인에서 직접 $y[n] = h[n] * x[n]$ 컨볼루션 계산하는 것보다 DFT를 이용해 주파수 도메인에서 곱셈 $Y[k] = H[k]X[k]$으로 처리하는 것이 FFT 알고리즘 덕분에 훨씬 빠르다. 그러나 실제 음성 신호 길이는 매우 길기 때문에 신호를 일정 길이의 블록으로 나누어 처리해야 한다. 이때 신호의 특정 프레임만을 남기기 위해서 윈도우가 필요하다.

$$x_r[n] = x[n]w[n - rL]$$

이때 윈도우 함수와의 곱셈이 주파수 도메인에서는 컨볼루션으로 작동하기 때문에, 윈도우의 길이와 종류에 따라 다른 종류의 왜곡이 생길 수 있다.

### A. Rectangular Window

사각 윈도우는 Main-lobe가 좁아 주파수 해상도 면에서는 우수하지만, Side-lobe가 커서 Spectral Leakage 문제가 발생한다.

### B. Hann Window

한 윈도우는 사각 윈도우보다 Side-lobe가 작아 Spectral Leakage 문제를 막을 수 있다.

$$w[n] = 0.5\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right)$$

## 2. Overlap-add Method

입력 신호 중 일부인 $x_r[n]$와 필터 $h[n]$를 컨볼루션하여 $y[n]$을 얻는 대신, 주파수 도메인에서 곱셈 후 iDFT하는 과정에서 발생할 수 있는 문제가 있다.

$$\mathrm{iDFT}(H[k]X[k]) = h[n] * x[n] * \sum_{r=-\infty}^{\infty} \delta[n - rN] \tag{1}$$

$$\mathrm{iDFT}(H[k]X[k]) \neq h[n] * x[n] \ \text{ where } N < L + P - 1 \tag{2}$$

### A. Circular Convolution

식 (1)에서 나타난 공식을 모듈러를 통해 정의한 것이 원형 컨볼루션이다.

$$h[n] \circledast x[n] = \sum_{k=-\infty}^{\infty} h[k]x\left[((n-k))_N\right]$$

$h[n]$의 길이가 $P$이고 $x[n]$의 길이가 $L$일 때 둘을 컨볼루션한 결과는 길이가 $L+P-1$이 된다. 이것을 주기가 $N$인 임펄스 트레인과 컨볼루션할 때 겹치는 구간이 발생한다면 원형 컨볼루션의 결과는 선형 컨볼루션과 달라진다. iDFT를 통해 왜곡되지 않은 출력을 얻기 위해서는 DFT size인 $N$을 $L+P-1$ 이상인 값으로 설정해야 한다.

### B. Overlap Methods

원형 컨볼루션으로 인해 발생하는 문제를 해결하기 위해 블록 컨볼루션에서 사용하는 두 가지 방법이 있다.

- **overlap-add:** 입력 신호를 길이 $L$의 블록으로 겹치지 않도록 분할한다. 각 블록을 적절한 DFT size $N = L+P-1$으로 DFT하기 위해 $L$개 샘플 이후로는 제로 패딩한다. 이렇게 얻은 길이 $N$의 DFT 결과를 필터 DFT 결과와 곱한 후 iDFT한다. iDFT한 결과를 이어붙일 때에, 출력 부분에서 겹치는 구간의 $P-1$개의 샘플을 더하여 최종 출력을 얻는다.
- **overlap-save:** 입력 신호를 길이 $P-1$개씩 겹쳐 길이 $N = L+P-1$의 블록으로 분할한다. 각 블록을 DFT size $N$으로 DFT하고, 필터 DFT 결과와 곱한 후 iDFT한다. 이때 iDFT 과정에서 맨 앞 $P-1$개 샘플은 원형 컨볼루션으로 인해 왜곡된 부분이므로 버리고, 나머지 $L$개의 출력 샘플만을 취해 겹침 없이 이어붙인다.

### C. Overlap-add Method

이번 실험에서 사용한 overlap-add method는 입력 신호를 자를 때 겹침을 허용했다.

- $L = 80$ (frame shift 샘플 수)
- $P = 11$ (필터의 길이)
- $N = 128$
  - **With Window:** 24개 샘플에 윈도잉 + 56개 샘플 추출 + 24개 샘플에 윈도잉 + 24개 제로 패딩
  - **Without Window:** 104개 샘플 추출 + 24개 제로 패딩

  $\Rightarrow N \geq L+P-1$ 만족

  $\Rightarrow i$-th frame의 첫 24개의 샘플은 $(i-1)$-th frame의 104개 샘플 중 마지막 24개 샘플

# II. 결과

## 1. 코드 링크

II-2에 기재된 결과를 얻은 코드와 그 결과로 생성된 .wav 파일들을 깃허브에서 확인할 수 있다.

26-Spring-DSSP/DSSP_PA_2 at main · Yejin-02/26-Spring-DSSP

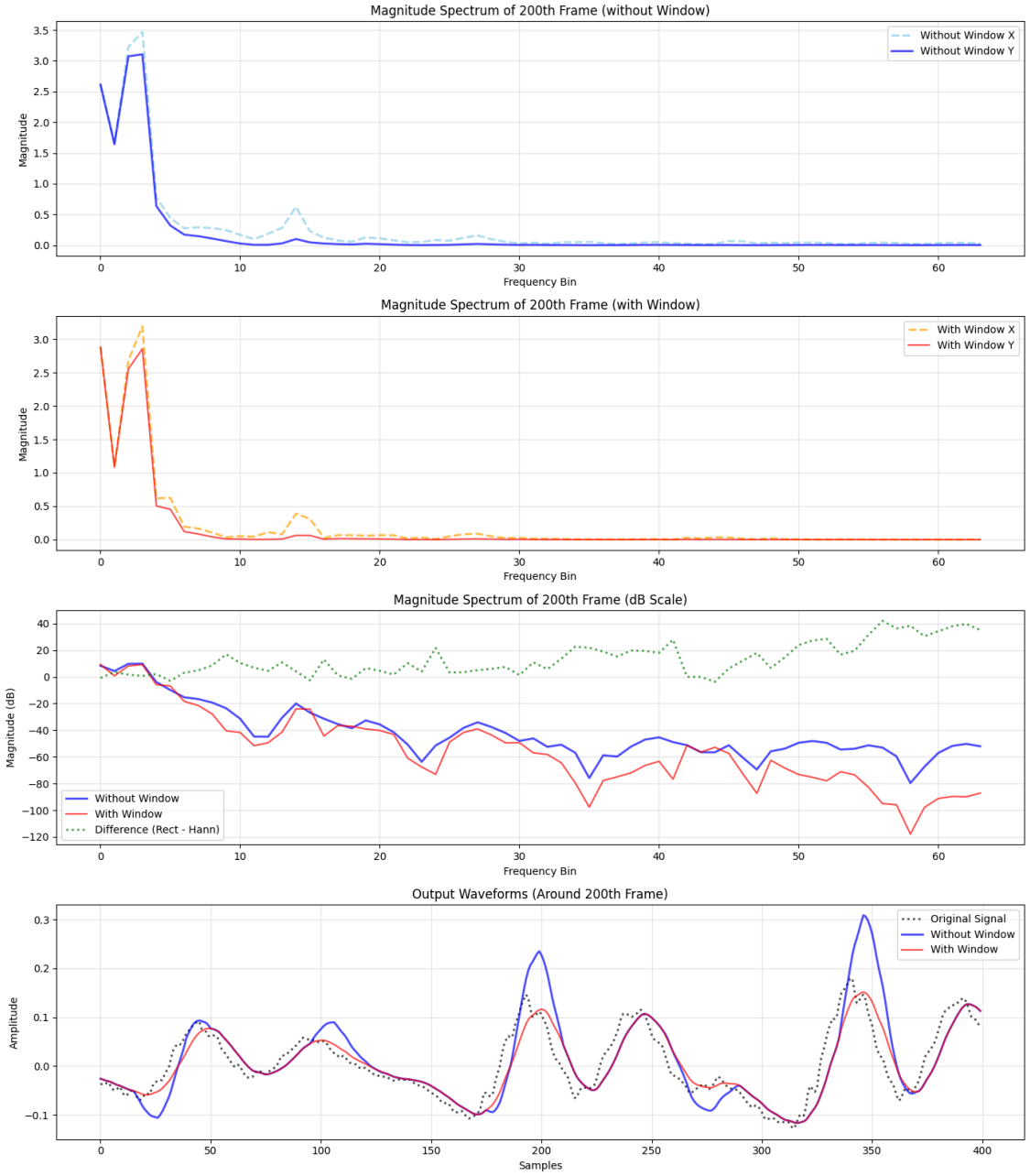
Contribute to Yejin-02/26-Spring-DSSP development by creating an account on GitHub.

https://github.com/Yejin-02/26-Spring-DSSP/tree/main/DSSP_PA_2

Yejin-02/**26-Spring-DSSP**

1 Contributor · 0 Issues · 0 Stars · 0 Forks

## 2. 결과 분석




## A. Linear Scale Spectrum

- 첫 번째 플롯은 윈도우를 사용하지 않았을 때 200번째 프레임에서의 X[k], Y[k]를 frequency bin에 따른 선형 스케일의 Magnitude로 나타낸 것이다.
- 두 번째 플롯은 윈도우를 사용했을 때 200번째 프레임에서의 X[k], Y[k]를 frequency bin에 따른 선형 스케일의 Magnitude로 나타낸 것이다.
- Low Pass Filter를 통과한 결과이기 때문에 두 경우 모두 입력보다 출력에서 고주파 성분이 줄어든 것을 확인할 수 있다.
- 그러나 `y_hann.wav` 와 `y_rect.wav` 를 직접 들었을 때 큰 차이가 느껴지는 것에 비해, 고주파 대역에서 구체적으로 어떤 차이가 나타나는지 확인할 수 없었다.
- 따라서 dB 스케일에서 그림을 추가로 확인하였다.

## B. dB Scale Spectrum

- `Y_rect_200[:N_fft//2]` 대신 `20 * np.log10(np.abs(Y_rect_200[:N_fft//2]) + 1e-10)` 를 통해 dB 스케일의 스펙트럼을 세 번째 플롯과 같이 확인하였다.
- 그 결과 Hann 윈도우를 사용한 경우와 사용하지 않은 경우, 고주파 대역에서 최대 40dB 이상의 차이가 발생하는 것을 확인할 수 있었다.
- 이러한 차이가 발생한 이유는, 시간 도메인에서 Rectangular Windowing을 사용하면 주파수 도메인에서 Sinc 함수를 컨볼루션 하는 역할을 하게 되는데, 이때 Sinc 함수의 Side-lobe로 인해 저주파 대역의 에너지가 고주파 대역으로 누수되는 현상이 발생했기 때문이다.
- 반면 Hann Windowing은 시간 도메인에서 양끝을 서서히 증가하고 감쇠시킨다. 이를 통해 주파수 도메인에서 Side-lobe를 크게 억제하기 때문에 고주파 대역에서 Spectral Leakage로 인한 왜곡을 막을 수 있다.
- 실제로 `y_hann.wav` 는 `y_rect.wav` 보다 훨씬 듣기에 깔끔했다.

## C. Waveform

- 시간 도메인에서는 두 종류의 출력이 원본 신호와 어떤 차이가 있는지 200번째 프레임 근처의 샘플들을 통해 네 번째 플롯과 같이 확인하였다.
- `y_hann.wav` 은 원 신호의 fine structure를 적절히 smoothing하여 envelope을 얻는 것을 확인했다.
- 반면 `y_rect.wav` 은 원 신호보다 과장된 peak를 나타내는 것을 확인했다. 길이가 104인 Hann 윈도우에서는 크로스 페이드 작용이 있기 때문에 윈도우 길이가 Frame Shift보다 큰 효과가 상쇄됐지만, 사각 윈도우에는 단순히 덧셈을 하는 것이 되므로 overlap-add할 때 겹치는 구간에서 진폭이 과장되는 현상이 발생한 것이다.
- 또한, 두 개의 파일 모두 원 신호보다 몇 개의 샘플만큼 delay가 발생했는데 이는 로우 패스 필터를 적용하는 과정에서 생긴 그룹 딜레이 때문이다.